**Kingdom of Saudi Arabia**
**Ministry of Higher Education**
**King Faisal University**
**College of Computer Sciences & Information Technology**


# Smart City Management system

## Database Concepts and Design Course Project

Year: 2025-26 Semester: I

| No. | Student Name | Student ID |
|---|---|---|
| **S1** | Abdulrahman Alrashidi | 221418666 |
| **S2** | Mohammed Alghafli | 222412271 |
| **S3** | Hadi Almubarak | 222426012 |
| **S4** | Abdullah Alghadeer | 222447516 |

**Instructor:** Dr. Mohammed Bonias

# Table of Contents

## Table of Figures

# 1. Introduction (case description)

The current rate at which cities are being urbanized has created a huge demand for intelligent solutions for efficiently managing the structure and resources as well as providing services for citizens. In this project report, the design and development process for Smart City Management System with Centralized Relational DBS is being described for better management and improved quality of life for citizens.

The case study is focused on the digitization of three critical areas within local governance: Incident Reporting, Parking Management, and IoT Sensor Data Monitoring. Typically, local governance bodies face inefficient storage and management of unconnected datasets, leading to inadequate response time to civic structural issues and inefficient use of parking capacity. In effect, the developed system would be able to efficiently connect Citizens, Locations, and City Assets.

It allows Citizens to make their contribution to the maintenance of the city by providing Reports about particular problems (like potholes, illegal parking, and malfunctioning street lights) at specific geographic locations. As far as the automation process is concerned, the database can manage a chain of IoT Sensors laid out across the city. The sensors use a combination approach: some sensors measure conditions such as temperature at specific street addresses, and the rest are allocated to specific Parking Spaces to determine their current occupancy rate.

With this information organized within a strong relational schema with proper constraints and foreign key associations, the Smart City Management System provides city managers with the ability to view the status of reports, the evaluation of parking capacity, and a historical log for later analysis based on sensor readings.

## 2. System Analysis

### A. List of users

- Citizen (General User)
- City Operator (Staff/Dispatcher)
- Data Analyst / City Planner
- System Administrator
- IoT Sensor (System Actor)

### B. List of main functions (at least three)

1. Manage Incident Lifecycle (Create, Update, Resolve)
2. Provide Real-time Parking Availability
3. Monitor Environment & Trigger Alerts

### C. List of main reports (at least three)

1. Incident Response Efficiency Report
2. Parking Occupancy & Hotspot Report
3. Environmental Hazard Report

## 3. Database Conceptual Design (ERD)

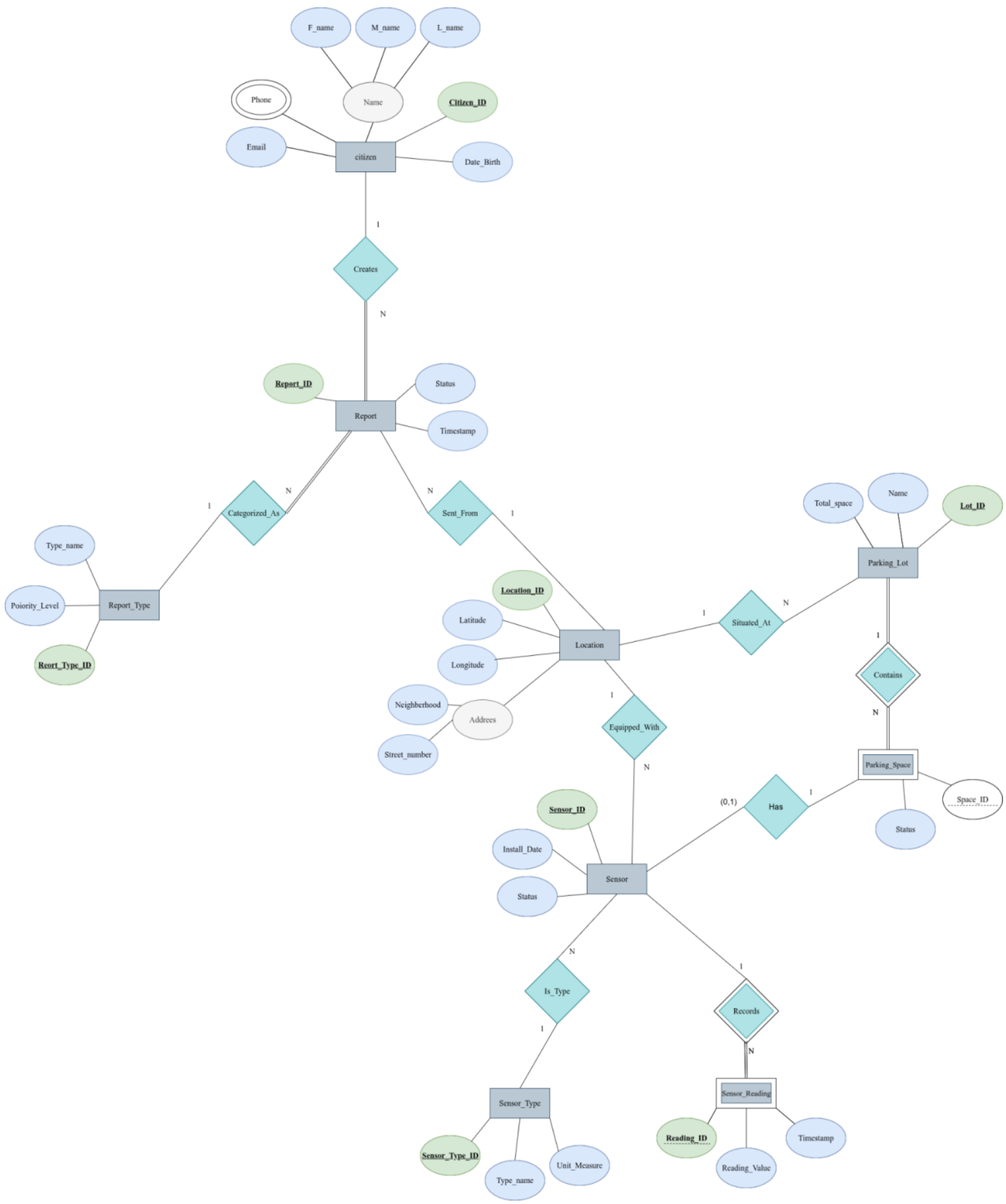


*Figure 1: Entity Relationship Diagram (ERD) for the Smart City Management System.*

# 4. Logical Design (Relational Schema)

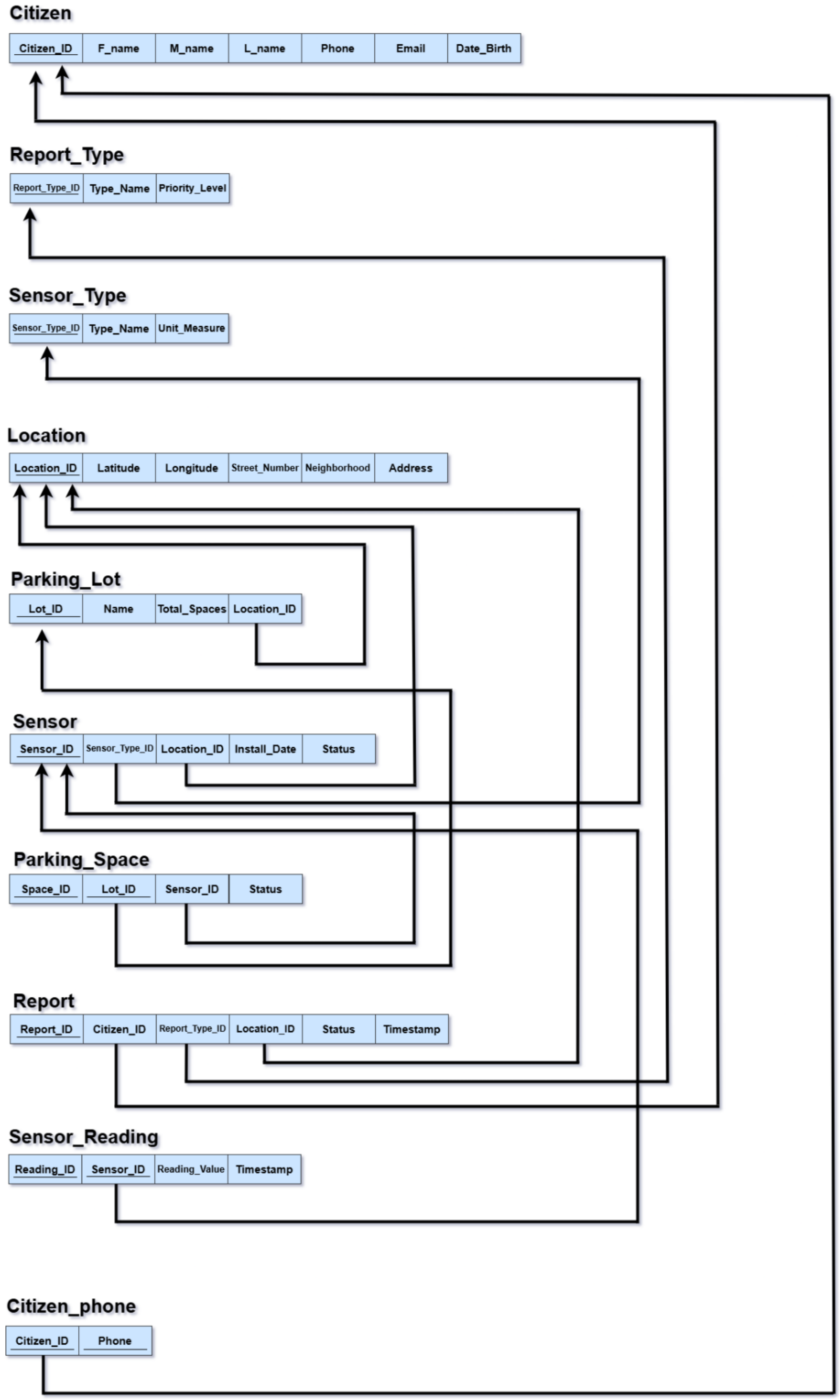


*Figure 2: Relational Database Schema illustrating tables and foreign key dependencies.*

## 5. Physical Design

```sql
CREATE TABLE Location (
   Location_ID NUMBER PRIMARY KEY,
   Neighborhood VARCHAR2(100),
   Street_Number VARCHAR2(50),
   Address VARCHAR2(255),
   Latitude NUMBER(10, 8),
   Longitude NUMBER(11, 8)
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| CITIZEN_ID | NUMBER | N | | 1 | | |
| F_NAME | VARCHAR2(50 BYTE) | Y | | | | |
| M_NAME | VARCHAR2(50 BYTE) | Y | | | | |
| L_NAME | VARCHAR2(50 BYTE) | Y | | | | |
| EMAIL | VARCHAR2(100 BYTE) | Y | | | | |
| DATE_BIRTH | DATE | Y | | | | |

```sql
CREATE TABLE Report_Type (
   Report_Type_ID NUMBER PRIMARY KEY,
   Type_Name VARCHAR2(50),
   Priority_Level NUMBER
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| REPORT_TYPE_ID | NUMBER | N | | 1 | | |
| TYPE_NAME | VARCHAR2(50 BYTE) | Y | | | | |
| PRIORITY_LEVEL | NUMBER | Y | | | | |

```sql
CREATE TABLE Citizen (
   Citizen_ID NUMBER PRIMARY KEY,
   F_Name VARCHAR2(50),
   M_Name VARCHAR2(50),
   L_Name VARCHAR2(50),
   Email VARCHAR2(100),
   Date_Birth DATE
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| CITIZEN_ID | NUMBER | N | | 1 | | |
| F_NAME | VARCHAR2(50 BYTE) | Y | | | | |
| M_NAME | VARCHAR2(50 BYTE) | Y | | | | |
| L_NAME | VARCHAR2(50 BYTE) | Y | | | | |
| EMAIL | VARCHAR2(100 BYTE) | Y | | | | |
| DATE_BIRTH | DATE | Y | | | | |

```sql
CREATE TABLE Sensor_Type (
```

```sql
    Sensor_Type_ID NUMBER PRIMARY KEY,
    Type_Name VARCHAR2(50),
    Unit_Measure VARCHAR2(20)
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| SENSOR_TYPE_ID | NUMBER | N | | 1 | | |
| TYPE_NAME | VARCHAR2(50 BYTE) | Y | | | | |
| UNIT_MEASURE | VARCHAR2(20 BYTE) | Y | | | | |

```sql
CREATE TABLE Citizen_Phone (
    Citizen_ID NUMBER,
    Phone_Number VARCHAR2(20),
    PRIMARY KEY (Citizen_ID, Phone_Number),
    FOREIGN KEY (Citizen_ID) REFERENCES Citizen(Citizen_ID) ON DELETE CASCADE
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| CITIZEN_ID | NUMBER | N | | 1 | | |
| PHONE_NUMBER | VARCHAR2(20 BYTE) | N | | 2 | | |

```sql
CREATE TABLE Parking_Lot (
    Lot_ID NUMBER PRIMARY KEY,
    Name VARCHAR2(100),
    Total_Spaces NUMBER,
    Location_ID NUMBER NOT NULL,
    FOREIGN KEY (Location_ID) REFERENCES Location(Location_ID)
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| LOT_ID | NUMBER | N | | 1 | | |
| NAME | VARCHAR2(100 BYTE) | Y | | | | |
| TOTAL_SPACES | NUMBER | Y | | | | |
| LOCATION_ID | NUMBER | N | | | | |

```sql
CREATE TABLE Report (
    Report_ID NUMBER PRIMARY KEY,
    Status VARCHAR2(20) DEFAULT 'Pending',
    Timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Citizen_ID NUMBER,
    Report_Type_ID NUMBER,
    Location_ID NUMBER,
    FOREIGN KEY (Citizen_ID) REFERENCES Citizen(Citizen_ID),
    FOREIGN KEY (Report_Type_ID) REFERENCES Report_Type(Report_Type_ID),
    FOREIGN KEY (Location_ID) REFERENCES Location(Location_ID)
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| REPORT_ID | NUMBER | N | | 1 | | |
| STATUS | VARCHAR2(20 BYTE) | Y | 'Pending' | | | |
| TIMESTAMP | TIMESTAMP(6) | Y | CURRENT_TIMESTAMP | | | |
| CITIZEN_ID | NUMBER | Y | | | | |
| REPORT_TYPE_ID | NUMBER | Y | | | | |
| LOCATION_ID | NUMBER | Y | | | | |

```
CREATE TABLE Parking_Space (
   Lot_ID NUMBER,
   Space_ID NUMBER,
   Status VARCHAR2(20) DEFAULT 'Empty',
   PRIMARY KEY (Lot_ID, Space_ID),
   FOREIGN KEY (Lot_ID) REFERENCES Parking_Lot(Lot_ID) ON DELETE CASCADE
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| LOT_ID | NUMBER | N | | 1 | | |
| SPACE_ID | NUMBER | N | | 2 | | |
| STATUS | VARCHAR2(20 BYTE) | Y | 'Empty' | | | |

```
CREATE TABLE Sensor (
   Sensor_ID NUMBER PRIMARY KEY,
   Status VARCHAR2(20) DEFAULT 'Active',
   Install_Date DATE,
   Sensor_Type_ID NUMBER NOT NULL,
   Location_ID NUMBER NOT NULL,
   Lot_ID NUMBER,
   Space_ID NUMBER,

   FOREIGN KEY (Sensor_Type_ID) REFERENCES Sensor_Type(Sensor_Type_ID),
   FOREIGN KEY (Location_ID) REFERENCES Location(Location_ID),
   FOREIGN KEY (Lot_ID, Space_ID) REFERENCES Parking_Space(Lot_ID, Space_ID) ON
DELETE SET NULL
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| SENSOR_ID | NUMBER | N | | 1 | | |
| STATUS | VARCHAR2(20 BYTE) | Y | 'Active' | | | |
| INSTALL_DATE | DATE | Y | | | | |
| SENSOR_TYPE_ID | NUMBER | N | | | | |
| LOCATION_ID | NUMBER | N | | | | |
| LOT_ID | NUMBER | Y | | | | |
| SPACE_ID | NUMBER | Y | | | | |

```
CREATE TABLE Sensor_Reading (
```

```
    Sensor_ID NUMBER,
    Reading_ID NUMBER,
    Reading_Value NUMBER(10, 2),
    Timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (Sensor_ID, Reading_ID),
    FOREIGN KEY (Sensor_ID) REFERENCES Sensor(Sensor_ID) ON DELETE CASCADE
);
```

| Column Name | Data Type | Nullable | Default | Primary Key | Comment | Identity |
|---|---|---|---|---|---|---|
| SENSOR_ID | NUMBER | N | | 1 | | |
| READING_ID | NUMBER | N | | 2 | | |
| READING_VALUE | NUMBER(10,2) | Y | | | | |
| TIMESTAMP | TIMESTAMP(6) | Y | CURRENT_TIMESTAMP | | | |

## 6. Populate Database

**Insertion for Location table**

```
INSERT INTO Location (Location_ID, Neighborhood, Street_Number, Address, Latitude, Longitude) VALUES (101, 'Downtown', '500', '500 Main St', 24.7136, 46.6753);
INSERT INTO Location (Location_ID, Neighborhood, Street_Number, Address, Latitude, Longitude) VALUES (102, 'University', '12', 'Uni Gate 3', 24.7200, 46.6800);
INSERT INTO Location (Location_ID, Neighborhood, Street_Number, Address, Latitude, Longitude) VALUES (103, 'Al Olaya', '88', '88 Olaya St', 24.6900, 46.6900);
```

| LOCATION_ID | NEIGHBORHOOD | STREET_NUMBER | ADDRESS | LATITUDE | LONGITUDE |
|---|---|---|---|---|---|
| 101 | Downtown | 500 | 500 Main St | 24.7136 | 46.6753 |
| 102 | University | 12 | Uni Gate 3 | 24.72 | 46.68 |
| 103 | Al Olaya | 88 | 88 Olaya St | 24.69 | 46.69 |

**Insertion for Report_Type table**

```
INSERT INTO Report_Type (Report_Type_ID, Type_Name, Priority_Level) VALUES (1, 'Pothole', 3);
INSERT INTO Report_Type (Report_Type_ID, Type_Name, Priority_Level) VALUES (2, 'Broken Light', 2);
INSERT INTO Report_Type (Report_Type_ID, Type_Name, Priority_Level) VALUES (3, 'Illegal Parking', 4);
```

| REPORT_TYPE_ID | TYPE_NAME | PRIORITY_LEVEL |
|---|---|---|
| 1 | Pothole | 3 |
| 2 | Broken Light | 2 |
| 3 | Illegal Parking | 4 |

**Insertion for Citizen table**

INSERT INTO Citizen (Citizen_ID, F_Name, M_Name, L_Name, Email, Date_Birth) VALUES (1001, 'Ahmed', 'Mohamed', 'Al-Saud', 'ahmed@ex.com', TO_DATE('1998-05-15', 'YYYY-MM-DD'));
INSERT INTO Citizen (Citizen_ID, F_Name, M_Name, L_Name, Email, Date_Birth) VALUES (1002, 'Sara', 'Ali', 'Al-Amri', 'sara@ex.com', TO_DATE('2001-11-20', 'YYYY-MM-DD'));

| CITIZEN_ID | F_NAME | M_NAME | L_NAME | EMAIL | DATE_BIRTH |
|---|---|---|---|---|---|
| 1001 | Ahmed | Mohamed | Al-Saud | ahmed@ex.com | 5/15/1998 |
| 1002 | Sara | Ali | Al-Amri | sara@ex.com | 11/20/2001 |

**Insertion for Sensor_Type table**

INSERT INTO Sensor_Type (Sensor_Type_ID, Type_Name, Unit_Measure) VALUES (10, 'Temperature', 'Celsius');
INSERT INTO Sensor_Type (Sensor_Type_ID, Type_Name, Unit_Measure) VALUES (20, 'Occupancy', 'Boolean');

| SENSOR_TYPE_ID | TYPE_NAME | UNIT_MEASURE |
|---|---|---|
| 10 | Temperature | Celsius |
| 20 | Occupancy | Boolean |

**Insertion for Citizen_Phone table**

INSERT INTO Citizen_Phone (Citizen_ID, Phone_Number) VALUES (1001, '0501234567');
INSERT INTO Citizen_Phone (Citizen_ID, Phone_Number) VALUES (1002, '0555551234');

| CITIZEN_ID | PHONE_NUMBER |
|---|---|
| 1001 | 0501234567 |
| 1002 | 0555551234 |

**Insertion for Parking_Lot table**

INSERT INTO Parking_Lot (Lot_ID, Name, Total_Spaces, Location_ID) VALUES (50, 'Main St Garage', 100, 101);
INSERT INTO Parking_Lot (Lot_ID, Name, Total_Spaces, Location_ID) VALUES (60, 'Campus Parking', 50, 102);

| LOT_ID | NAME | TOTAL_SPACES | LOCATION_ID |
|---|---|---|---|
| 50 | Main St Garage | 100 | 101 |
| 60 | Campus Parking | 50 | 102 |

**Insertion for Report table**

INSERT INTO Report (Report_ID, Status, Timestamp, Citizen_ID, Report_Type_ID, Location_ID) VALUES (5001, 'Pending', TIMESTAMP '2025-10-01 08:30:00', 1001, 1, 103);

| | REPORT_ID | STATUS | TIMESTAMP | CITIZEN_ID | REPORT_TYPE_ID | LOCATION_ID |
|---|---|---|---|---|---|---|
| | 5001 | Pending | 01-OCT-25 08.30.00.000... | 1001 | 1 | 103 |

**Insertion for Parking_Space table**
INSERT INTO Parking_Space (Lot_ID, Space_ID, Status) VALUES (50, 1, 'Occupied');
INSERT INTO Parking_Space (Lot_ID, Space_ID, Status) VALUES (50, 2, 'Empty');
INSERT INTO Parking_Space (Lot_ID, Space_ID, Status) VALUES (60, 1, 'Occupied');

| | LOT_ID | SPACE_ID | STATUS |
|---|---|---|---|
| | 50 | 1 | Occupied |
| | 50 | 2 | Empty |
| | 60 | 1 | Occupied |

**Insertion for Sensor table**
INSERT INTO Sensor (Sensor_ID, Status, Install_Date, Sensor_Type_ID, Location_ID, Lot_ID, Space_ID)
VALUES (901, 'Active', TO_DATE('2024-01-01', 'YYYY-MM-DD'), 10, 103, NULL, NULL);
INSERT INTO Sensor (Sensor_ID, Status, Install_Date, Sensor_Type_ID, Location_ID, Lot_ID, Space_ID)
VALUES (902, 'Active', TO_DATE('2024-02-15', 'YYYY-MM-DD'), 20, 101, 50, 1);

| | SENSOR_ID | STATUS | INSTALL_DATE | SENSOR_TYPE_ID | LOCATION_ID | LOT_ID | SPACE_ID |
|---|---|---|---|---|---|---|---|
| | 901 | Active | 1/1/2024 | 10 | 103 | | |
| | 902 | Active | 2/15/2024 | 20 | 101 | 50 | 1 |

**Insertion for Sensor_Reading table**
INSERT INTO Sensor_Reading (Sensor_ID, Reading_ID, Reading_Value, Timestamp)
VALUES (901, 1, 35.5, TIMESTAMP '2025-10-05 12:00:00');

INSERT INTO Sensor_Reading (Sensor_ID, Reading_ID, Reading_Value, Timestamp)
VALUES (902, 1, 1, TIMESTAMP '2025-10-05 08:00:00');

| | SENSOR_ID | READING_ID | READING_VALUE | TIMESTAMP |
|---|---|---|---|---|
| | 901 | 1 | 35.5 | 05-OCT-25 12.00.00.000000000 PM |
| | 902 | 1 | 1 | 05-OCT-25 08.00.00.000000000 AM |

## 7. Queries

1. Show All Registered Citizens
SELECT * FROM Citizen;

2. Filter Locations by Neighborhood
SELECT Location_ID, Address, Street_Number
FROM Location
WHERE Neighborhood = 'Downtown';

3. List High-Capacity Parking Lots

```
SELECT Name, Total_Spaces
FROM Parking_Lot
WHERE Total_Spaces > 50;
```

4. Find "Pending" Reports

```
SELECT Report_ID, Timestamp, Status
FROM Report
WHERE Status = 'Pending';
```

5. List All Temperature Sensors

```
SELECT Sensor_Type_ID, Type_Name, Unit_Measure
FROM Sensor_Type
WHERE Type_Name = 'Temperature';
```

6. Find readings that happened after a specific date.

```
SELECT Reading_ID, Reading_Value, Timestamp
FROM Sensor_Reading
WHERE Timestamp > TIMESTAMP '2025-10-01 00:00:00'
ORDER BY Timestamp DESC;
```

Results Explain Describe Saved SQL History

| READING_ID | READING_VALUE | TIMESTAMP |
|---|---|---|
| 1 | 35.5 | 05-OCT-25 12.00.00.000000 PM |
| 1 | 1 | 05-OCT-25 08.00.00.000000 AM |

2 rows returned in 0.02 seconds Download

7. The "Full Report" View Instead of seeing Citizen_ID (1001), show the actual name ("Ahmed").

```
SELECT
    r.Report_ID,
    c.F_Name || ' ' || c.M_Name || ' ' || c.L_Name AS Citizen_Name,
    r.Status,
    r.Timestamp
FROM Report r
JOIN Citizen c ON r.Citizen_ID = c.Citizen_ID;
```

Results Explain Describe Saved SQL History

| REPORT_ID | CITIZEN_NAME | STATUS | TIMESTAMP |
|---|---|---|---|
| 5001 | Ahmed Mohamed Al-Saud | Pending | 01-OCT-25 08.30.00.000000 AM |

1 rows returned in 0.03 seconds Download

8. Sensor Inventory with Location. Show where each sensor is installed and what type it is.

```
SELECT
    s.Sensor_ID,
```

```
    st.Type_Name,
    l.Address,
    l.Neighborhood
FROM Sensor s
JOIN Sensor_Type st ON s.Sensor_Type_ID = st.Sensor_Type_ID
JOIN Location l ON s.Location_ID = l.Location_ID;
```

Results Explain Describe Saved SQL History

| SENSOR_ID | TYPE_NAME | ADDRESS | NEIGHBORHOOD |
|---|---|---|---|
| 901 | Temperature | 88 Olaya St | Al Olaya |
| 902 | Occupancy | 500 Main St | Downtown |

2 rows returned in 0.00 seconds Download

9. Count how many occupied spaces are in each parking lot right now.

```
SELECT
    pl.Name AS Parking_Lot_Name,
    COUNT(ps.Space_ID) AS Occupied_Spaces
FROM Parking_Space ps
JOIN Parking_Lot pl ON ps.Lot_ID = pl.Lot_ID
WHERE ps.Status = 'Occupied'
GROUP BY pl.Name;
```

Results Explain Describe Saved SQL History

| PARKING_LOT_NAME | OCCUPIED_SPACES |
|---|---|
| Campus Parking | 1 |
| Main St Garage | 1 |

2 rows returned in 0.04 seconds Download

10. Find the names of all citizens who have reported a Pothole.

```
SELECT F_Name, L_Name, Email
FROM Citizen
WHERE Citizen_ID IN (
    SELECT Citizen_ID
    FROM Report
    WHERE Report_Type_ID = (
        SELECT Report_Type_ID
        FROM Report_Type
        WHERE Type_Name = 'Pothole'
    )
);
```

Results Explain Describe Saved SQL History

| F_NAME | L_NAME | EMAIL |
|---|---|---|
| Ahmed | Al-Saud | ahmed@ex.com |

1 rows returned in 0.06 seconds Download

# 8. Application Development

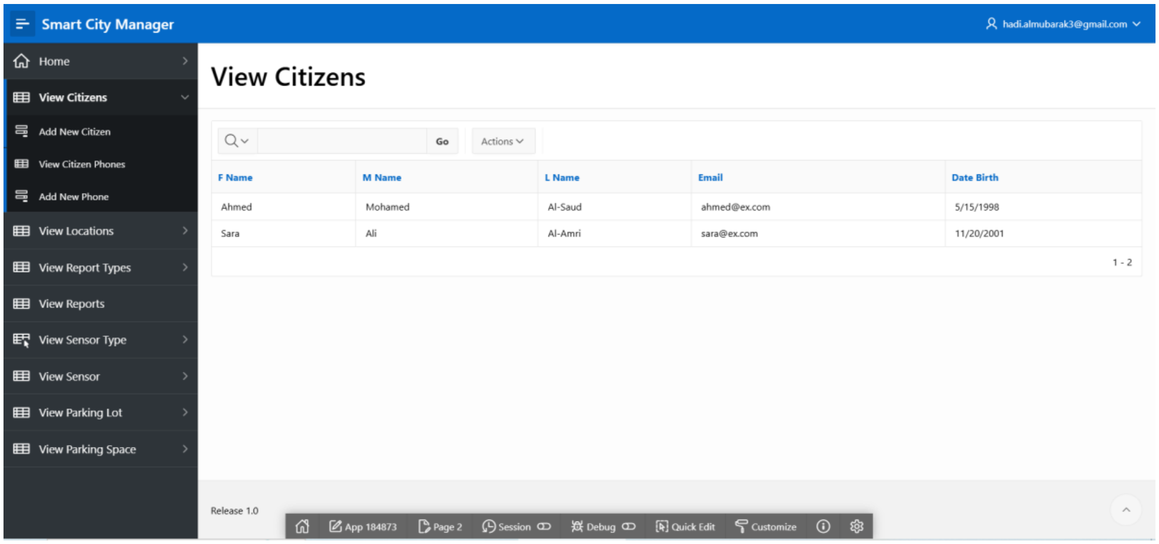


*Figure 3: View Citizens Page*

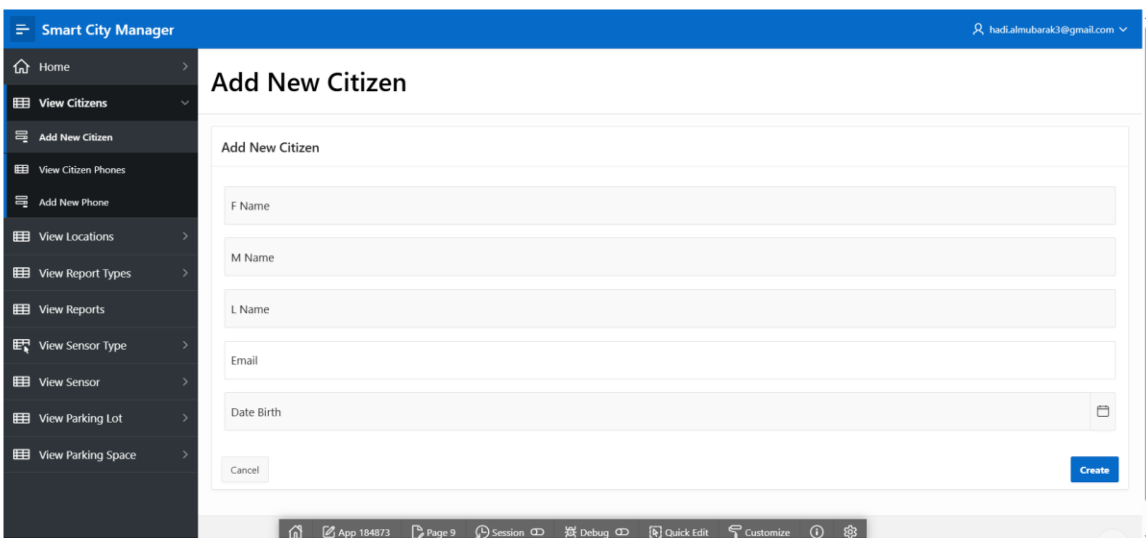


*Figure 4: Add New Citizen Page*

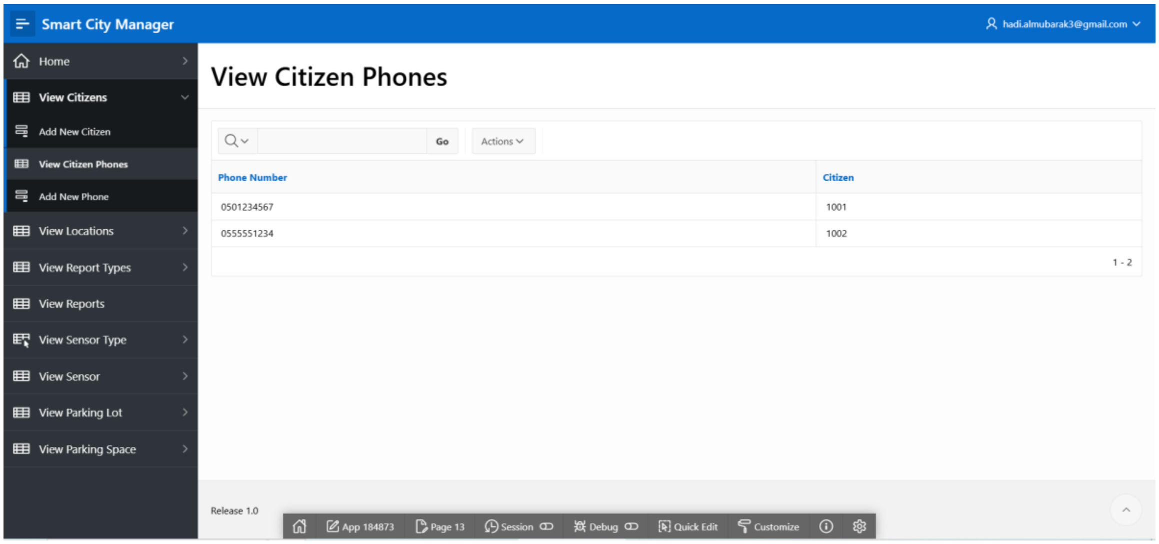


*Figure 5: View Citizen Phones*

Figures 3, 4 and 5: Allows administrators to view, search, and manage citizen records, including registering new users and updating their contact details.

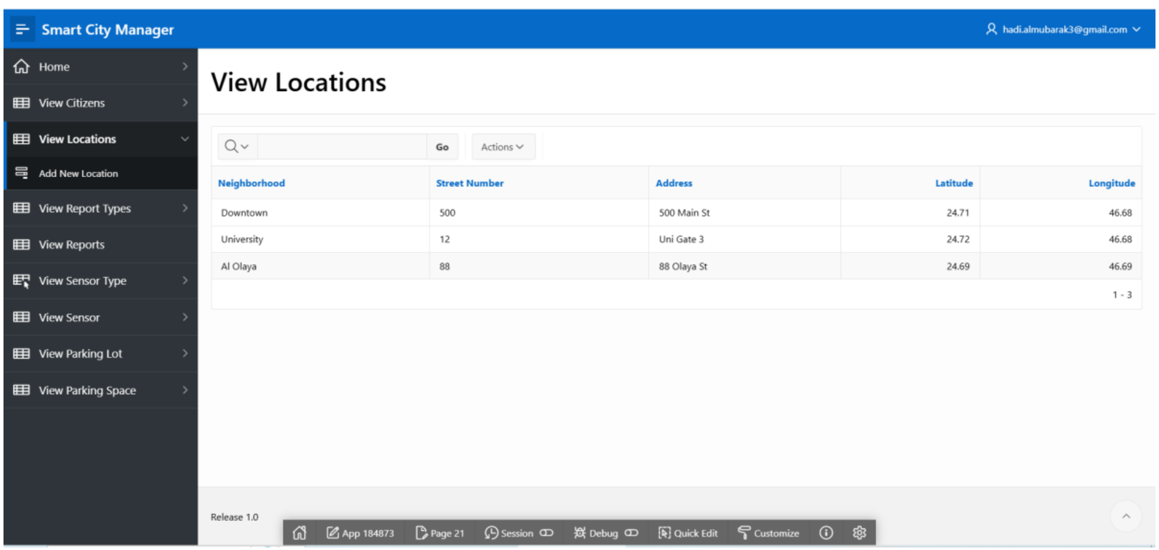

*Figure 6: View Locations*

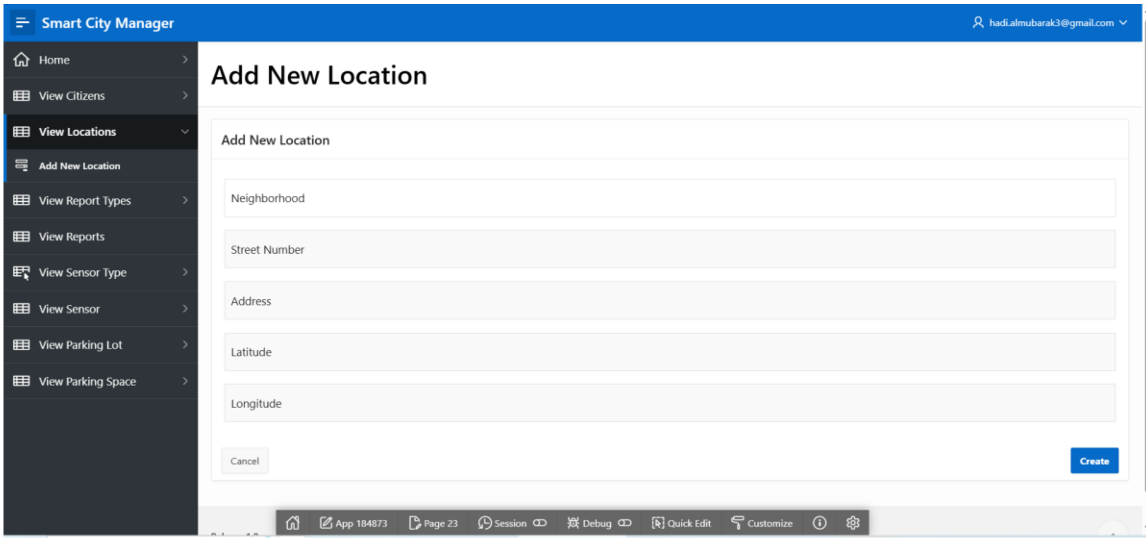


*Figure 7: Add New Location*

Figures 6, 7: Facilitates the management of city infrastructure points by storing address details and GPS coordinates for every monitored site.

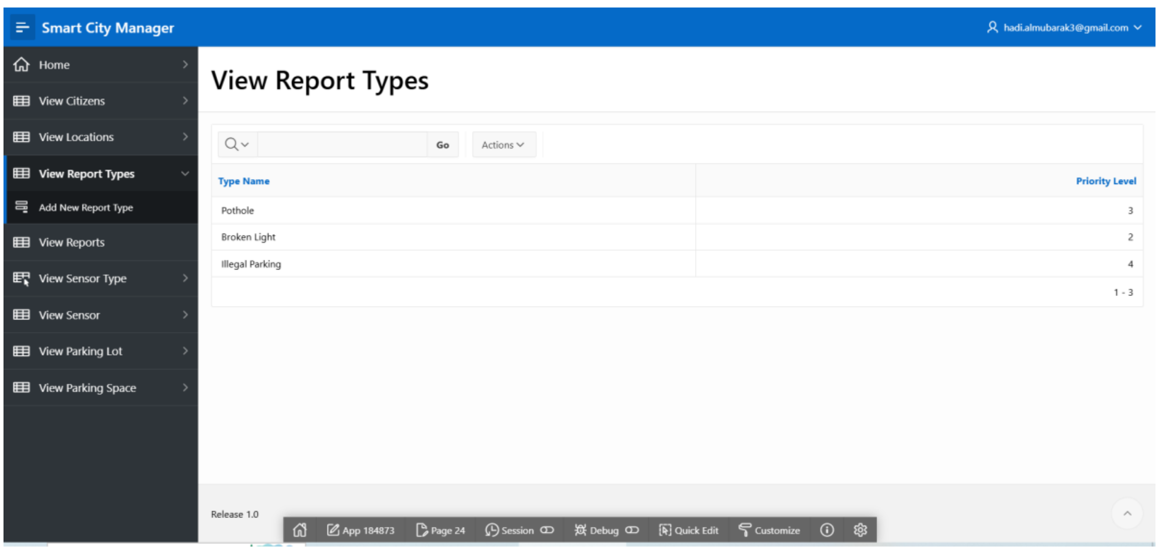


*Figure 8 : View Report Types*

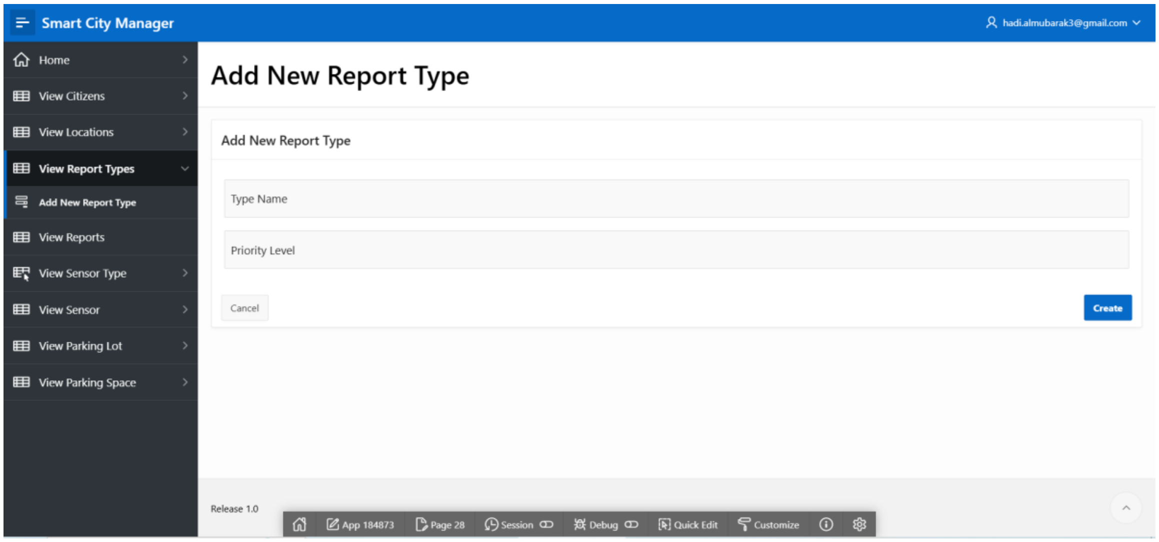


*Figure 9 : Add New Report Type*

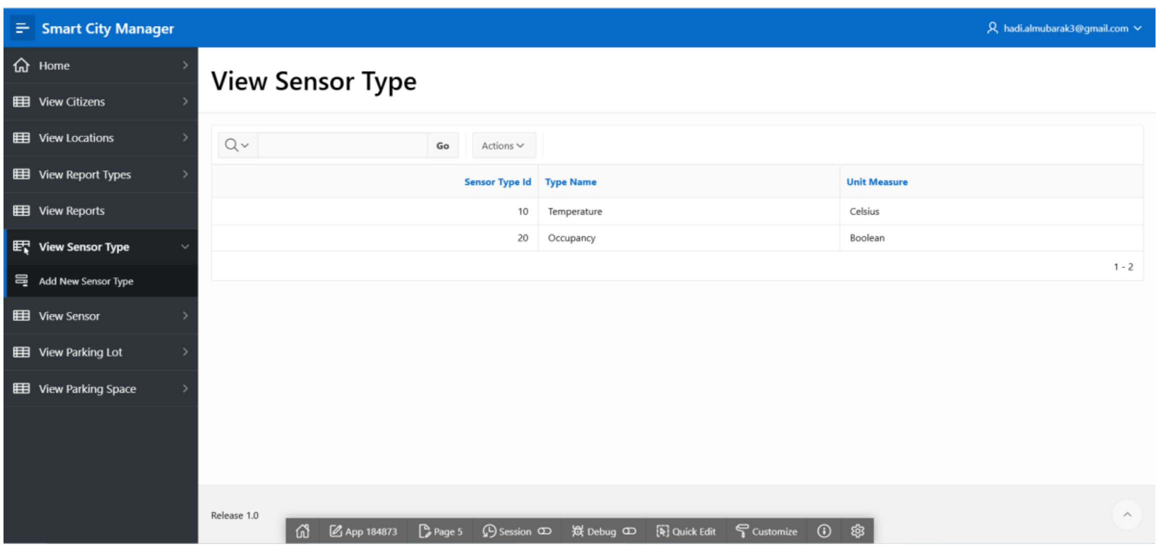


*Figure 10 : View Sensor Type*

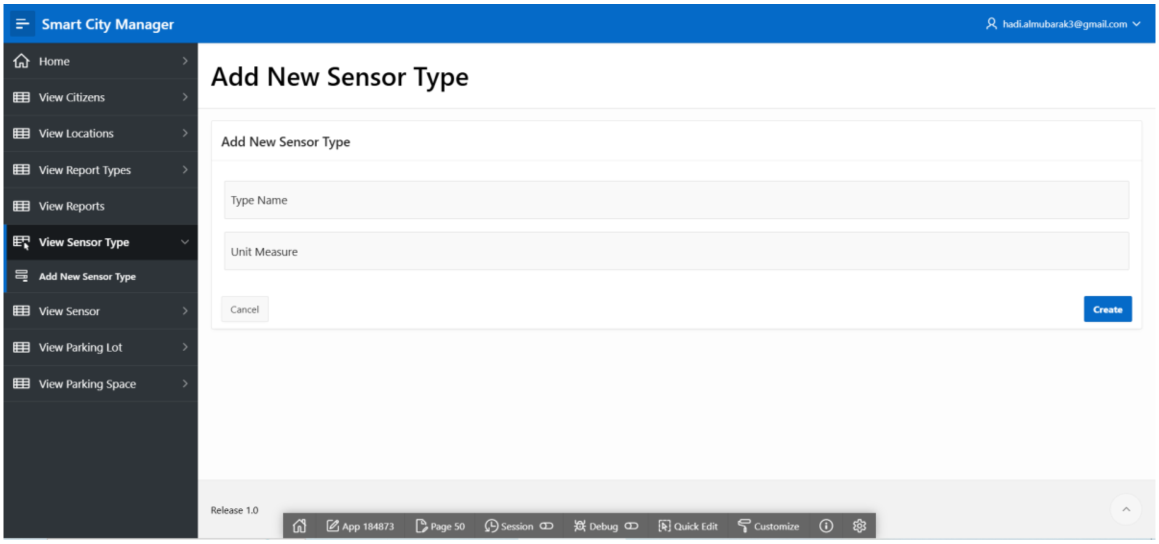


*Figure 11 : Add New Sensor Type*

Figures 8, 9, 10 and 11: Managed interfaces for defining standard categories (e.g., "Pothole", "Temperature") to ensure consistent data entry across the system.

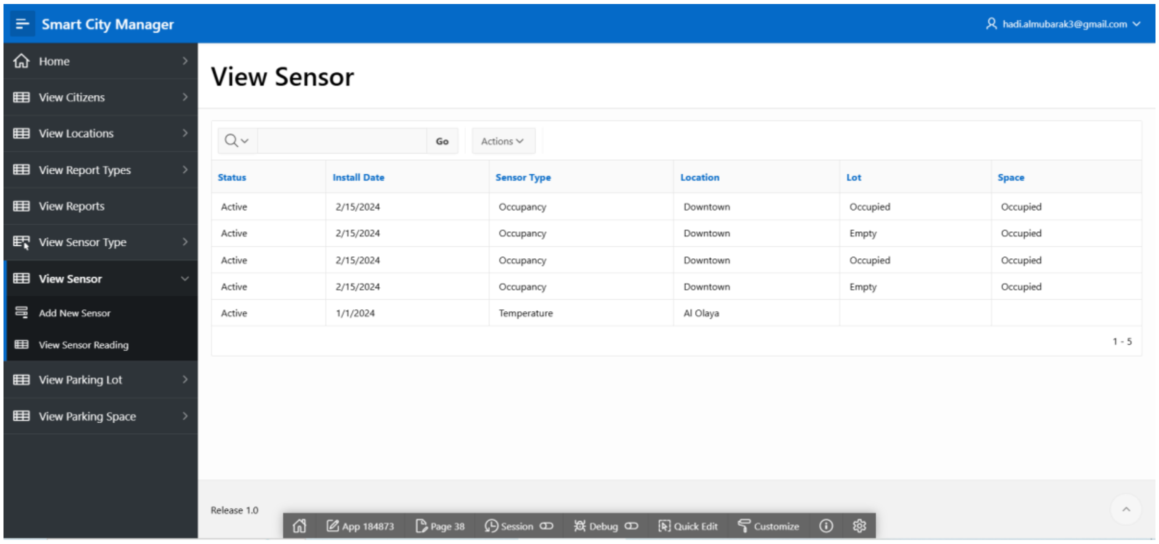


*Figure 12 : Sensors Page tracks the lifecycle of IoT devices installed across the city, recording their type, operational status, and specific installation location.*

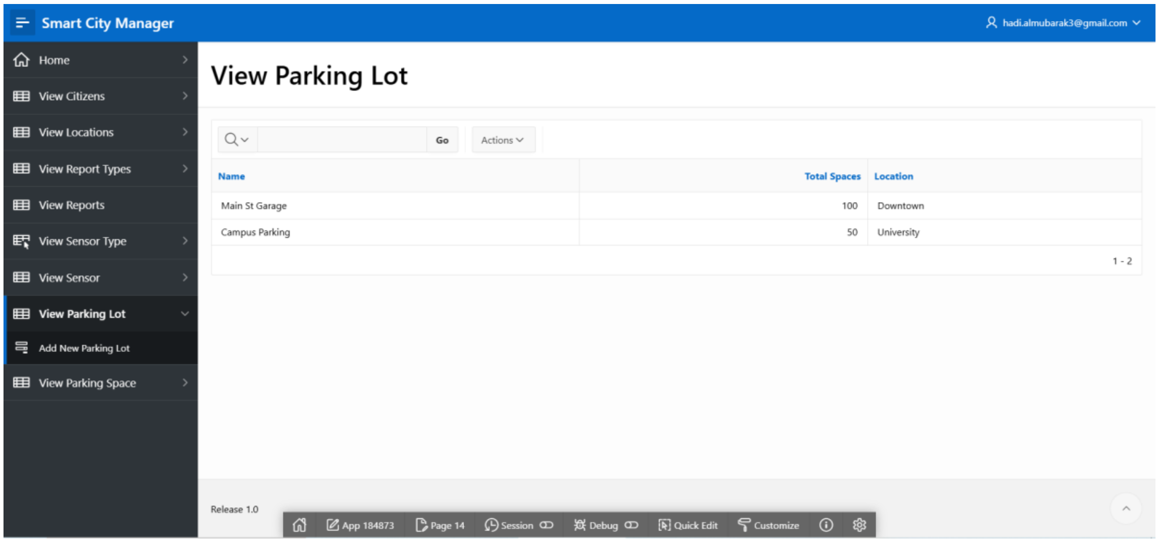

*Figure 13 : View Parking Lot*

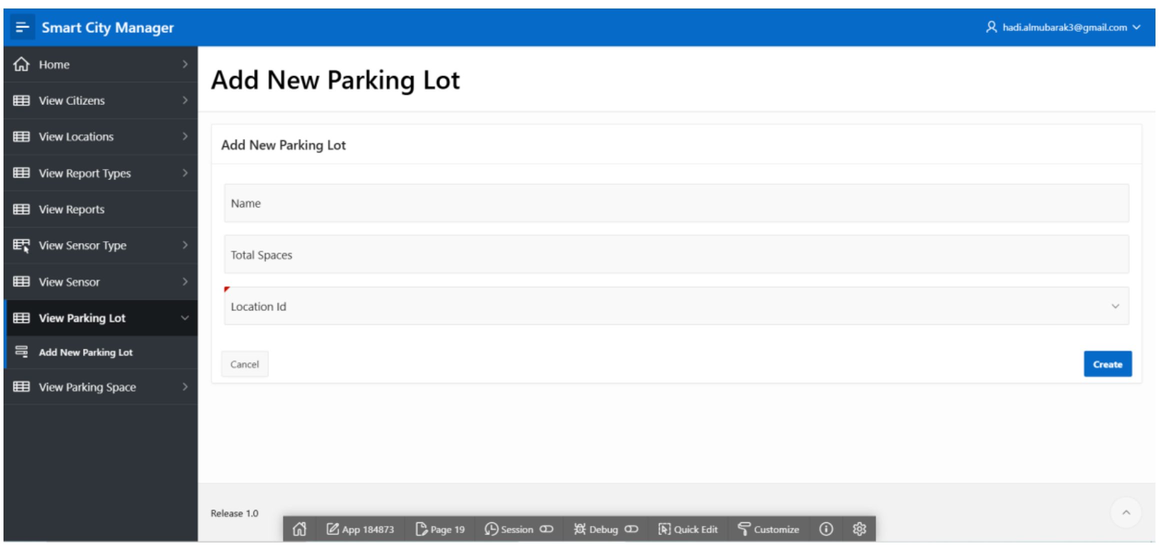

*Figure 14 : Add New Parking Lot*

Figures 13, 14 : Enables the configuration of parking facilities, allowing users to define lot names, capacities, and their geographical associations.

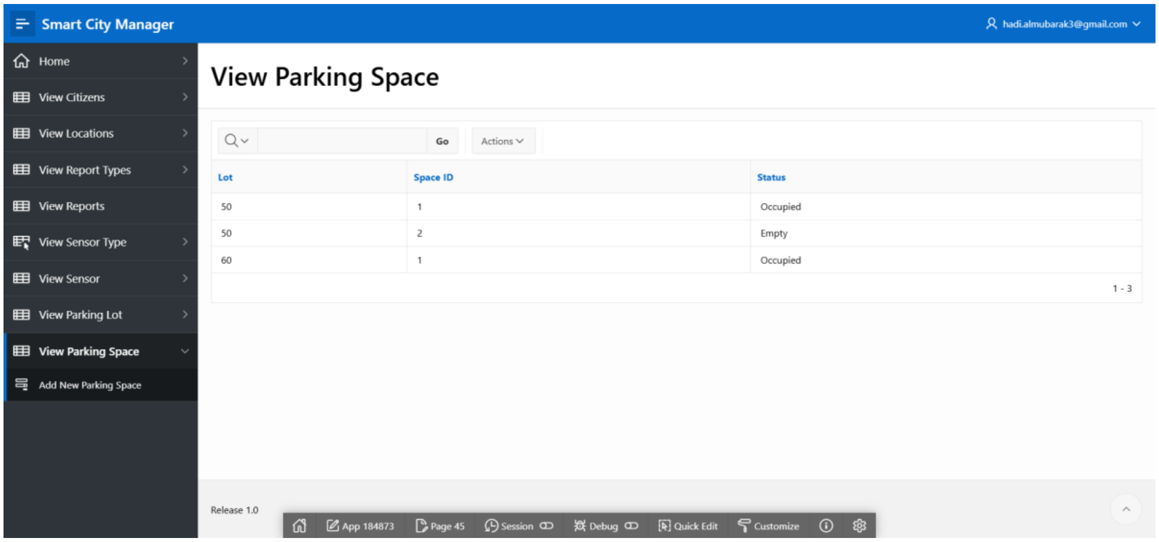


*Figure 15 : Parking Spaces Page manages the inventory of individual parking spots within specific lots, tracking unique identifiers and their current occupancy status.*

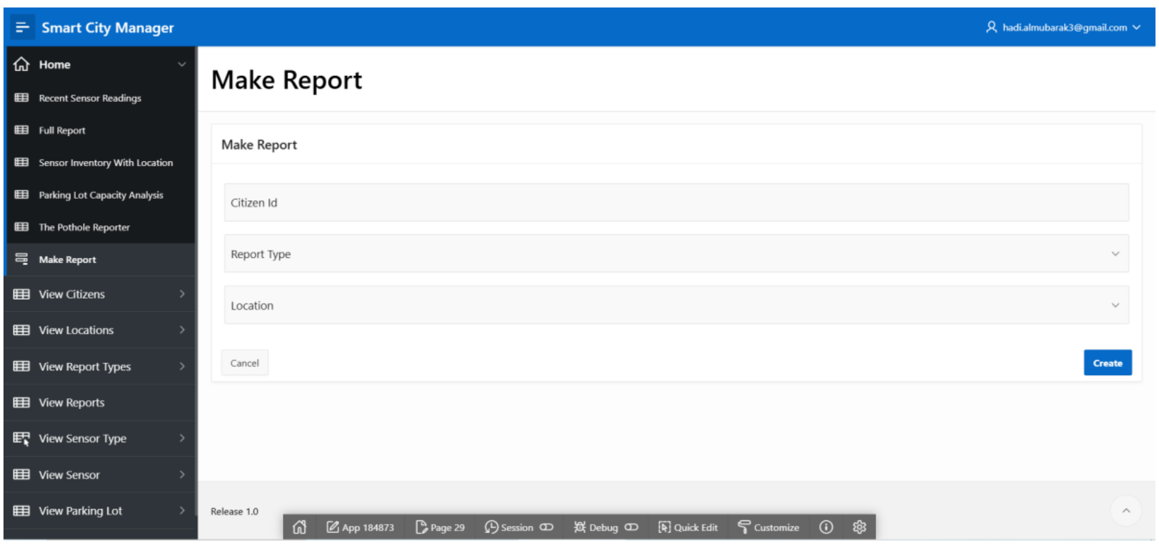


*Figure 16 : This interface allows citizens or administrators to log new incidents by selecting a predefined Report Type and the specific Location of the issue. The form automatically captures the timestamp and sets the initial status to 'Pending' to ensure a stan*